Gradually Retrofitting Assurance into Systems Software

A Separation-Logic Approach

Zain K Aamer, **Rini Banerjee**, **Hiroyuki Katsura**, David Kaloper-Meršinjak, Dimitrios J Economou, Kayvan Memarian, Dhruv Makwana, Neel Krishnaswami, Benjamin C Pierce, Christopher Pulte, Peter Sewell

Theory and Practice of Static Analysis Workshop
Rennes, France
12 January 2026

UNIVERSITY OF CAMBRIDGE
Penn University of Pennsylvania
UNIVERSITY OF OXFORD

Motivation

<u>We want bug-free software!</u>

How can we make software more reliable? Big variety of possible approaches

- Push-button analysis tools with more generic, **implicit** specifications (e.g. Infer, Valgrind)

- Traditional formal verification via richer, **explicit** specifications

Motivation

<u>We want bug-free software!</u>

How can we make software more reliable? Big variety of possible approaches

- Push-button analysis tools with more generic, **implicit** specifications (e.g. Infer, Valgrind)
- Traditional formal verification via richer, **explicit** specifications
  - Historically ‘all-or-nothing’, with proof as only end-goal
  - Requires user to write *intermediate* specifications (e.g. loop invariants)

Where to start?!

The problem

Writing specifications is **hard in itself!**

- Using feedback from failed proof attempts to guide (intermediate) specification-writing and -fixing is tough
  - Requires specialist expertise
  - Proof attempts themselves can be time-consuming to run

$\Longrightarrow$ **slow feedback loop**

The problem

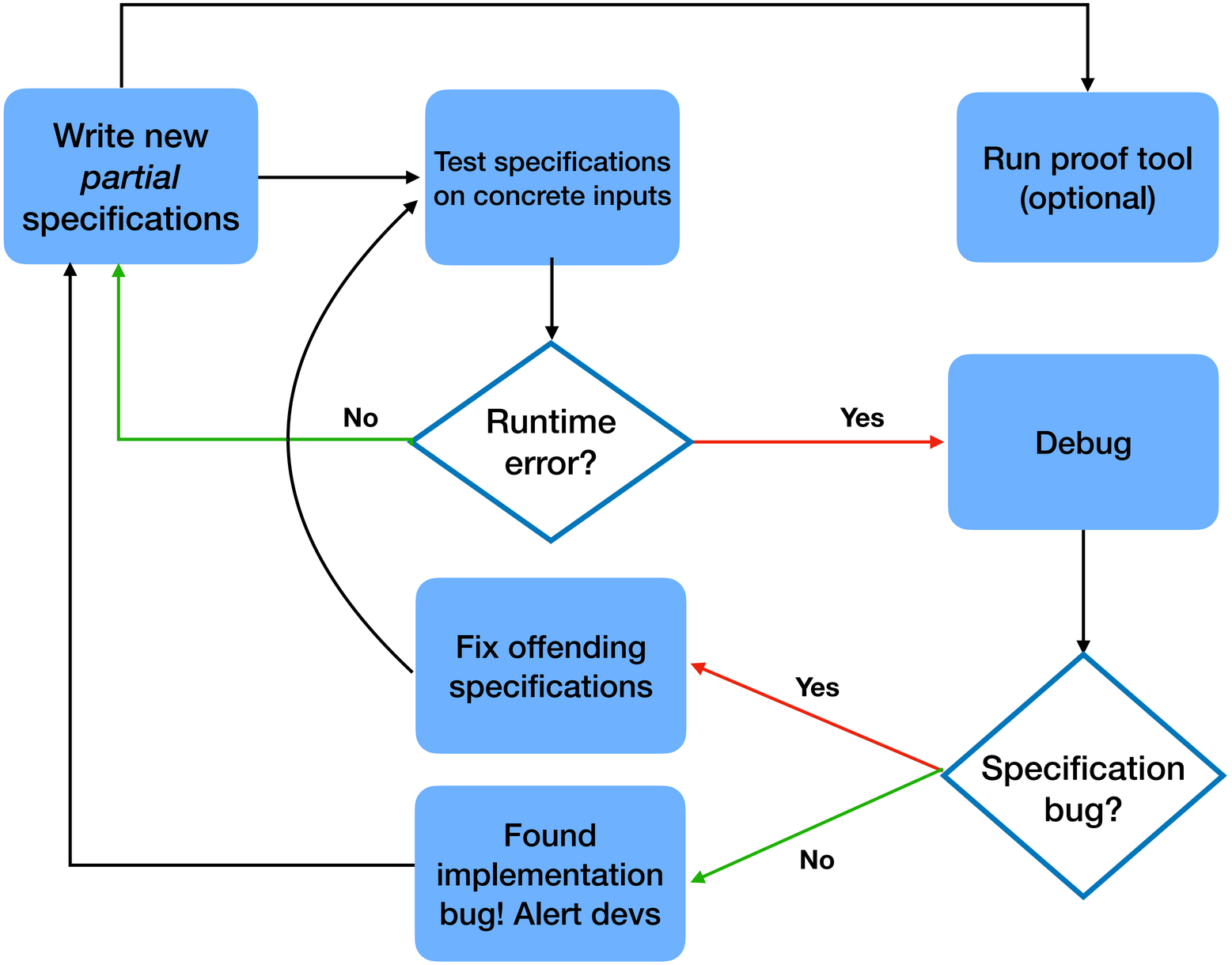
Writing specifications is **hard in itself!**

- Testing specifications could ease the process by flagging specification bugs quickly; this includes
  - **Runtime testing** the specification properties themselves, and
  - Generating **suitable random inputs** to exercise the runtime specifications
- … but most work in this space focuses on a single isolated approach/tool
- …and the testing and verification communities have historically been largely disconnected

We propose an ***incremental*** workflow that:

- ✓ builds upon the standard **code-test-debug** cycle
- ✓ puts **specification-writing and -testing** at its core
- ✓ combines **testing and proof** under a **single framework**

Code-Specify-Test-Debug-Prove



Code-Specify-Test-Debug-Prove

Code-Specify-Test-Debug-Prove

for **systems software written in C**

using **separation logic** to capture its complex ownership patterns

$$p \mapsto v$$

$$A * B$$

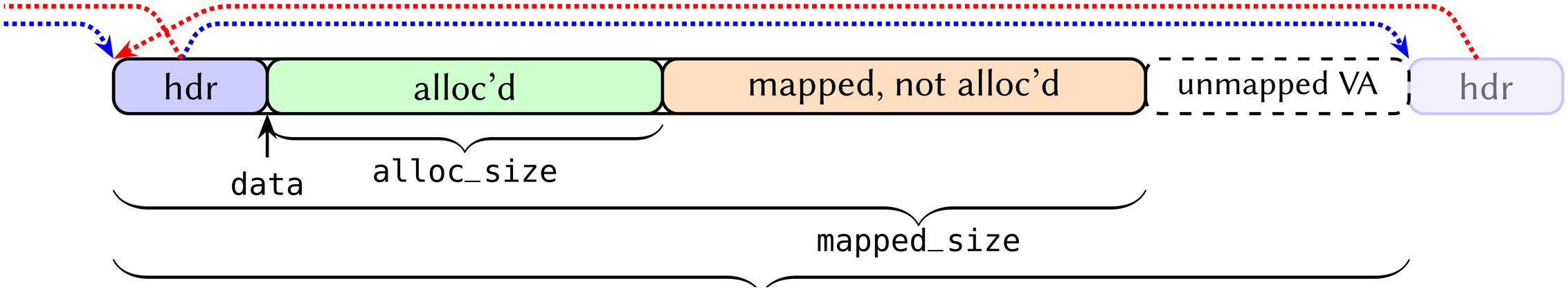
Using the CN separation-logic testing and verification toolchain

Case study: pKVM hyp allocator

- pKVM: real world hypervisor for Android developed by Google
  - Enforces memory isolation between the Android Linux kernel and guest VMs handling sensitive data
  - Runs at ARM Exception Level 2 (EL2)
  - Mostly written in C
- *hyp* allocator: heap allocator for pKVM like malloc/free in libc

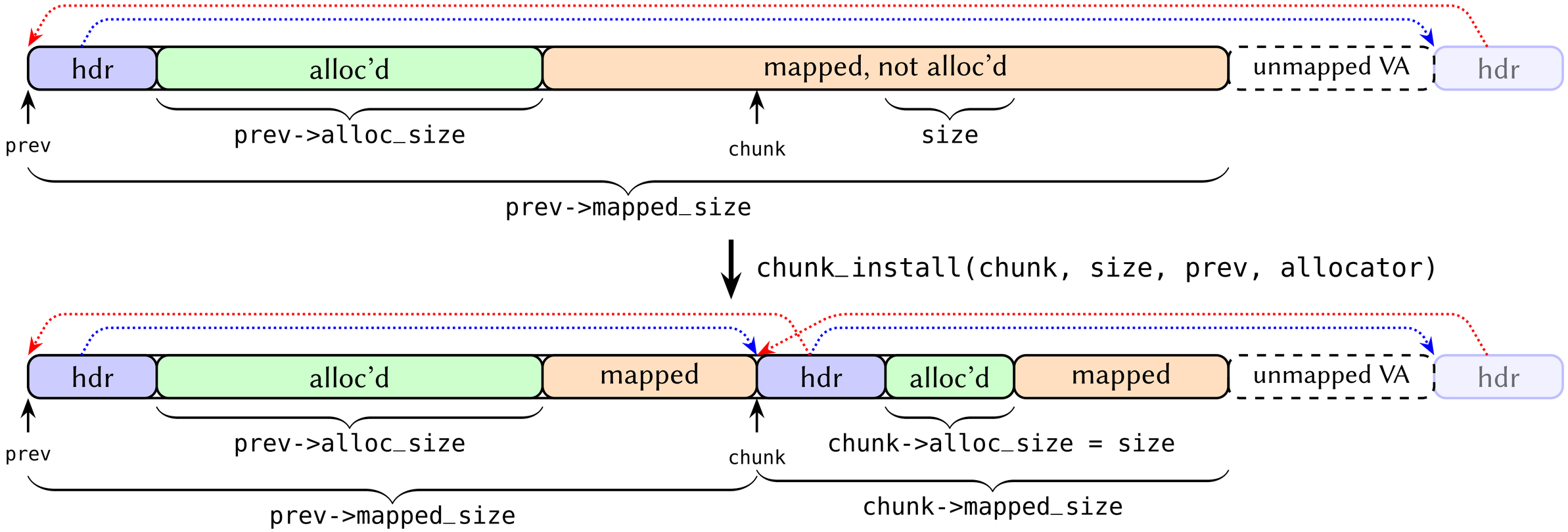
Case study: pKVM hyp allocator

- Memory chunks are managed using doubly-linked lists



Overview of `chunk_install`

- Taking a *chunk* starting at **`prev`** and an *address* for the new chunk **in the prev chunk**, **`chunk_install`** inserts the chunk by updating their metadata



Candidate spec for `chunk_install` in CN

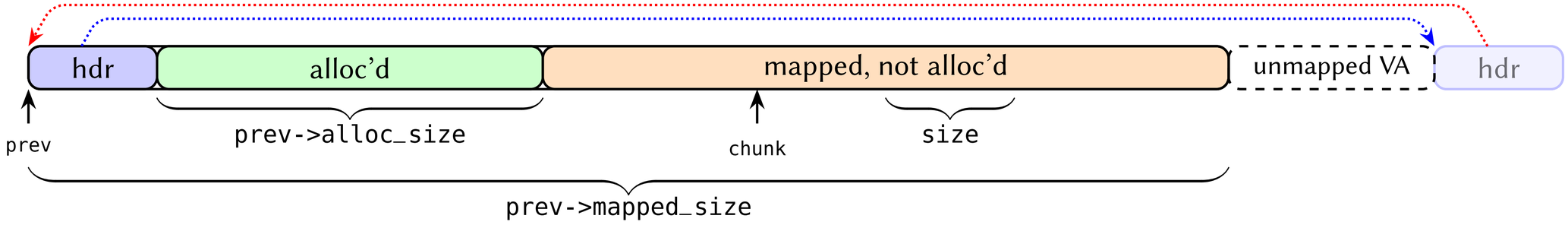
```
static int chunk_install(struct chunk_hdr *chunk, size_t size,
     struct chunk_hdr *prev, struct hyp_allocator *allocator)
/*@
requires
  take ha = OwnAllocator(allocator);
  take P = OwnChunk(prev, ha);

  (u64)chunk + hdr_size() + size
       <= (u64)prev + (u64)P.mapped_size;
ensures
  ... omitted...
@*/
```

Precondition of chunk_install

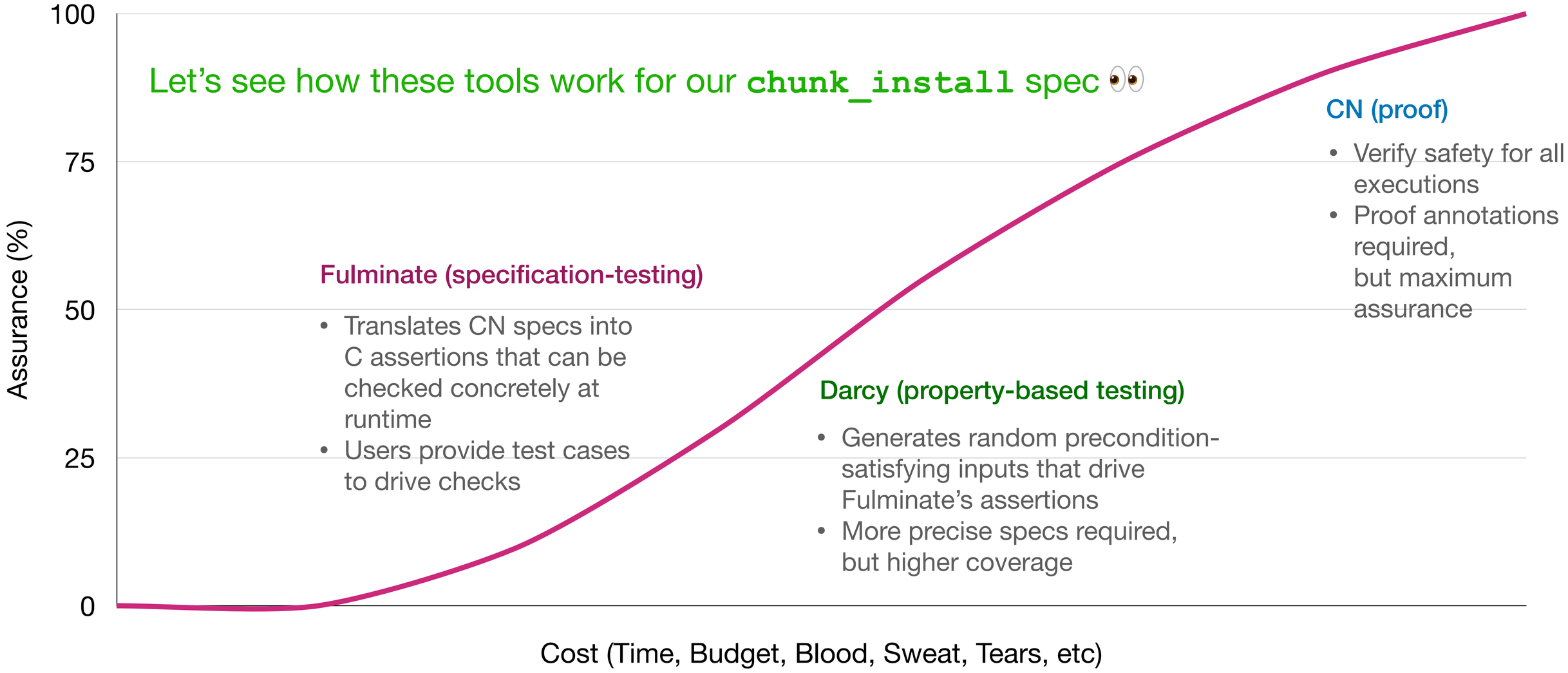
Ownership predicates (~ points-to)

Semicolons = Separating conjunctions

Logical constraints



Gradual assurance via testing



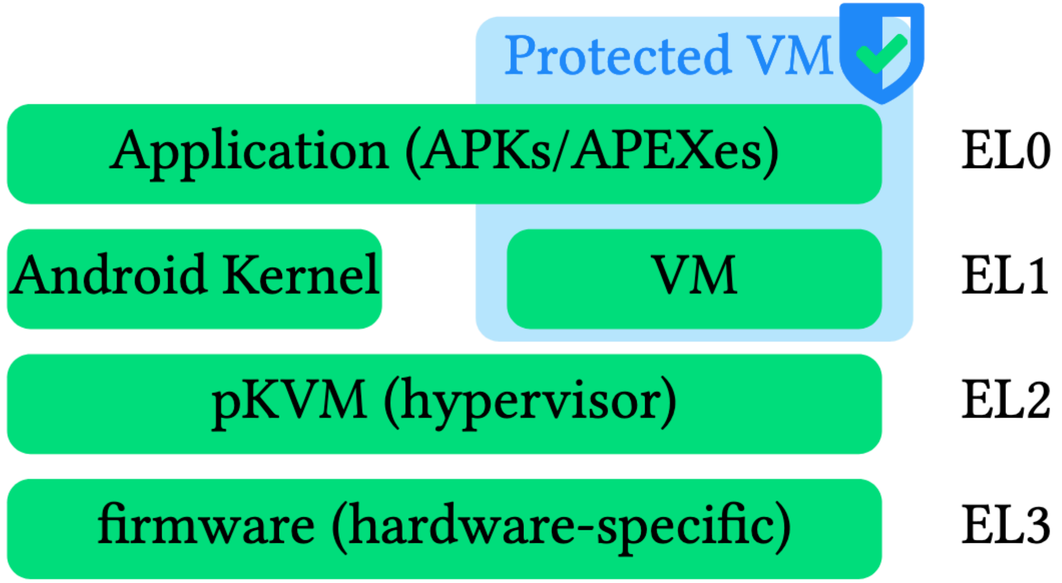
Demo!

Code-Specify-Test-Debug-Prove for the hyp allocator

- Applied this testing-specs-first workflow to a standalone version of the hyp allocator
  - Found **20** *specification* bugs and **4** *implementation* bugs
- Verified two main APIs: **`hyp_alloc`** and **`hyp_free`**
  - **913** lines of C code
  - **1846** lines of specifications and **735** lines of proof annotations

pKVM runs at EL2!

- Bare-metal environment
  - No malloc/free, no third-party libraries
- Preprocessed hyp allocator in Linux build environment has **150x** expansion, > **130,000 LoC**
  - Compared to 1.28x expansion for standalone allocator



We ran Fulminate on this*, and ran the Fulminate-instrumented hyp allocator *in situ* at EL2 :-)

Figure taken from Ghost in the Android Shell: Pragmatic Test-oracle Specification of a Production Hypervisor. Kayvan Memarian, Ben Simner, David Kaloper-Meršinjak, Thibaut Pérami, Peter Sewell. SOSP 2025.

*with some caveats I'm happy to discuss after this talk :-)

…and everything we needed to do to make this all work

- Various improvements to Fulminate
  - Support for more CN+C features, improved performance
- Implementation of new Darcy PBT tool
  - Uses SMT solver instead of backtracking to produce inputs for a wider range of preconditions
- Support for CN ghost arguments
- Improvements to CN proof performance
- *Lots* of low-level engineering to make Fulminate work *in situ* at EL2

Conclusion

- Testing specifications is ***really useful*** and more people should do it
- Writing good specifications is somewhat of an art, and testing partially-written specifications helps to significantly refine that process
- Being able to test and prove programs using the **same toolchain** is really helpful
  - Existing work: Frama-C provides a single framework for *Hoare-logic* specification testing, proof and static analysis of C programs
- The stepping-stone from toy examples to real-world case studies is **enormous** and requires a huge (and underappreciated) amount of engineering

CN: Verifying Systems C Code with Separation-Logic Refinement Types

CHRISTOPHER PULTE, University of Cambridge, UK
DHRUV C. MAKWANA, University of Cambridge, UK
THOMAS SEWELL, University of Cambridge, UK
KAYVAN MEMARIAN, University of Cambridge, UK
PETER SEWELL, University of Cambridge, UK
NEEL KRISHNASWAMI, University of Cambridge, UK

Fulminate: Testing CN Separation-Logic Specifications in C

RINI BANERJEE, KAYVAN MEMARIAN, DHRUV MAKWANA, CHRISTOPHER PULTE, NEEL KRISHNASWAMI, and PETER SEWELL, University of Cambridge, UK

Separation logic has become an important tool for formally capturing and reasoning about the ownership patterns of imperative programs, originally for paper proof, and now the foundation for industrial static analyses and multiple proof tools. However, there has been very little work on program *testing* of separation-logic specifications in concrete execution. At first sight, separation-logic formulas are hard to evaluate in reasonable time, with their implicit quantification over heap splittings, and other explicit existentials.